시스템 아키텍처

202344078 최예준 · 실제 코드 구조 기반 설계 문서

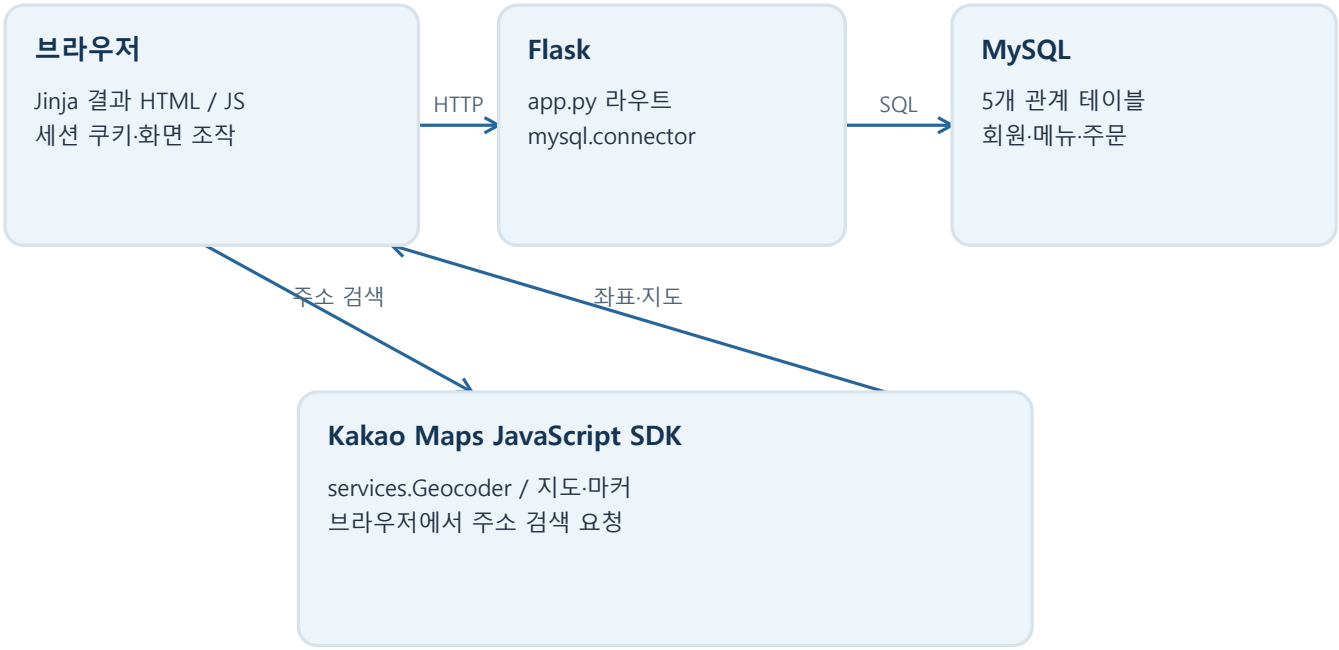


그림 1. 브라우저는 Flask에서 HTML을 받은 후 Kakao SDK를 통해 주소 좌표와 지도를 처리한다. 지도 API 요청을 Flask가 대신 중계하는 구조는 아니다.

1. 구성 요소와 책임

계층	구현 요소	역할
화면	templates/ 11개 HTML	폼 입력·Jinja 출력·JavaScript 지도 및 상태 처리
서버	app.py	인증·검색·장바구니·주문·통계·주소 라우트
영속 데이터	MySQL / DB SQL	회원·음식점·메뉴·주문·항목
외부 연동	Kakao Maps SDK services	주소 → 좌표, 지도·마커·정보창

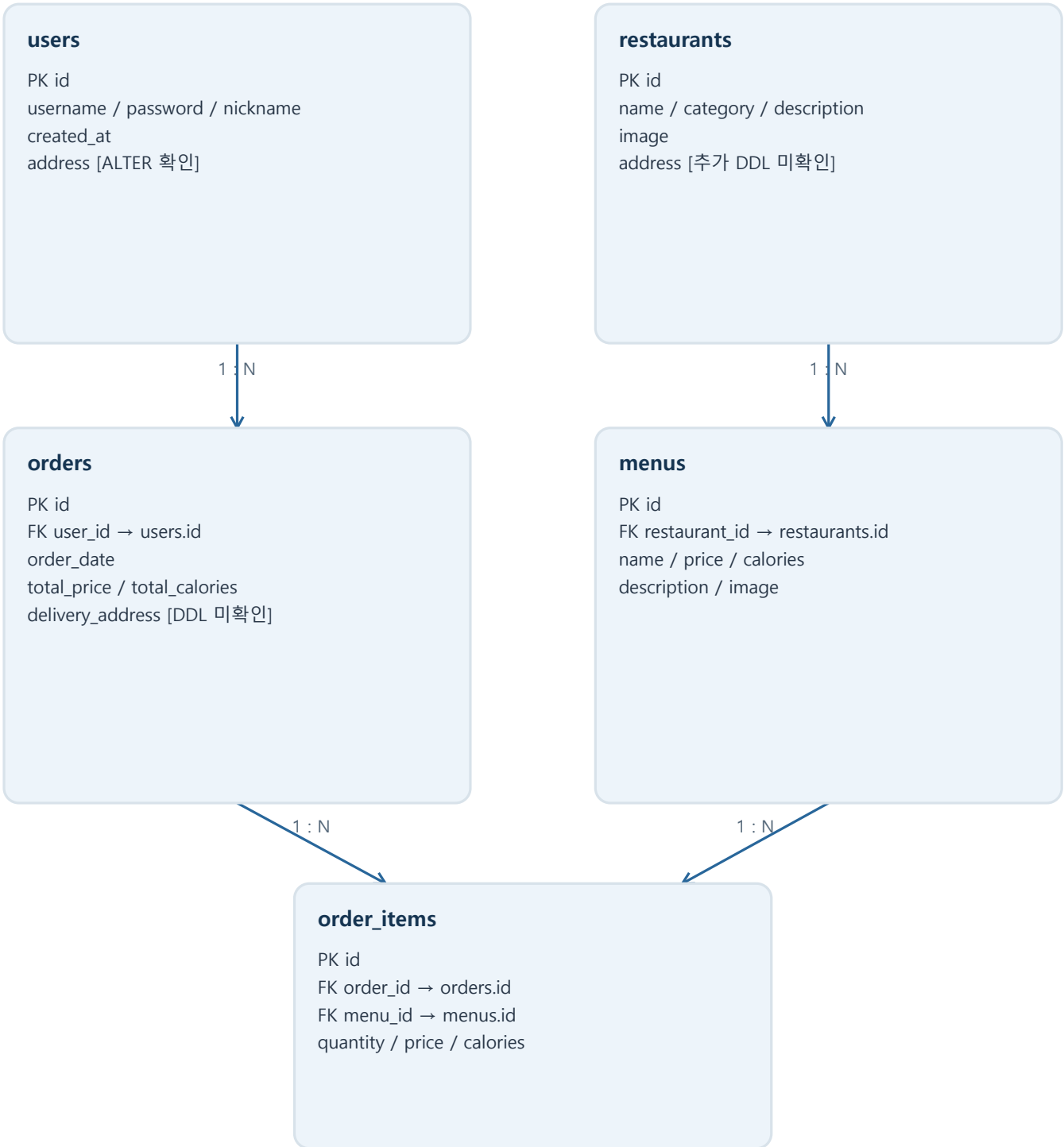
2. 설정 및 배포 경계

DB 접속 정보와 카카오 JavaScript 키는 환경변수에서 읽는다. SDK 키는 브라우저로 전달되므로 서버 전용 비밀키와 성격이 다르며, 사용 도메인 제한 등 운영 설정이 필요하다. 세션 비밀값은 현재 고정값으로 코드에 있으므로 외부화가 필요하다.

GitHub Pages는 정적 발표 HTML 용도다. 기본 Flask 세션은 서명된 클라이언트 쿠키이며 암호화된 저장소가 아니다. cart를 계정별 DB 테이블로 보관하지 않는다.

ERD: 5개 테이블의 관계

그림 2. PK/FK 표시는 저장된 schema.sql의 선언 기준이다.



실선 1:N은 초기 DDL에 FK 선언이 확인된 관계다. 현재 운영 DB에 적용된 제약은 직접 확인하지 않았다. address 확장 컬럼은 표시한 근거 수준으로 구분한다.

논리적 연결: orders → order_items → menus → restaurants. 주문과 음식점 사이에 직접 FK는 없다. 전체 자료형·NULL 조건은 분석 문서의 데이터 사전을 참조한다.

가상 주문 처리 흐름

그림 3. 실제 /order 함수의 검사·저장 순서를 반영한다.



3. 트랜잭션 및 기록 보존

주문 헤더와 항목 INSERT 후 한 번 commit한다. 정상 처리에서는 사용자 주소와 단가·칼로리가 함께 저장된다. DB 예외 발생 시 명시적인 rollback 및 공통 연결 정리는 구현되어 있지 않다.

조회되지 않는 메뉴는 건너뛴다. 유효 메뉴가 전혀 없으면 0원 주문이 생성될 수 있으며 별도 차단은 없다. 메뉴 합계와 항목 저장 사이의 동시 변경 잠금, 명시적 rollback도 없다.

배달현황 및 지도 처리

그림 4. 비동기 좌표 처리와 시간 기반 상태 갱신은 별도로 동작한다.



4. 시간 기반 가상 상태

order_date와 브라우저 현재 시각의 차이를 계산한다. 0~19초 주문 접수, 20~39초 음식 준비중, 40~59초 기사 배정, 60~79초 배달중, 80초 이후 완료로 표시한다. 1초마다 화면을 갱신하며 실제 배달 이벤트를 수신하지 않는다.

상태는 브라우저 시계·시간대에 의존한다. 지도 전체 실패 시 기본 범위를 유지한다. SDK 로드 실패의 별도 오류 처리와 미응답 콜백의 타임아웃은 없다.

주요 화면 및 URL 라우팅

그림 5. 기본 화면 이동과 서버 엔드포인트 대응

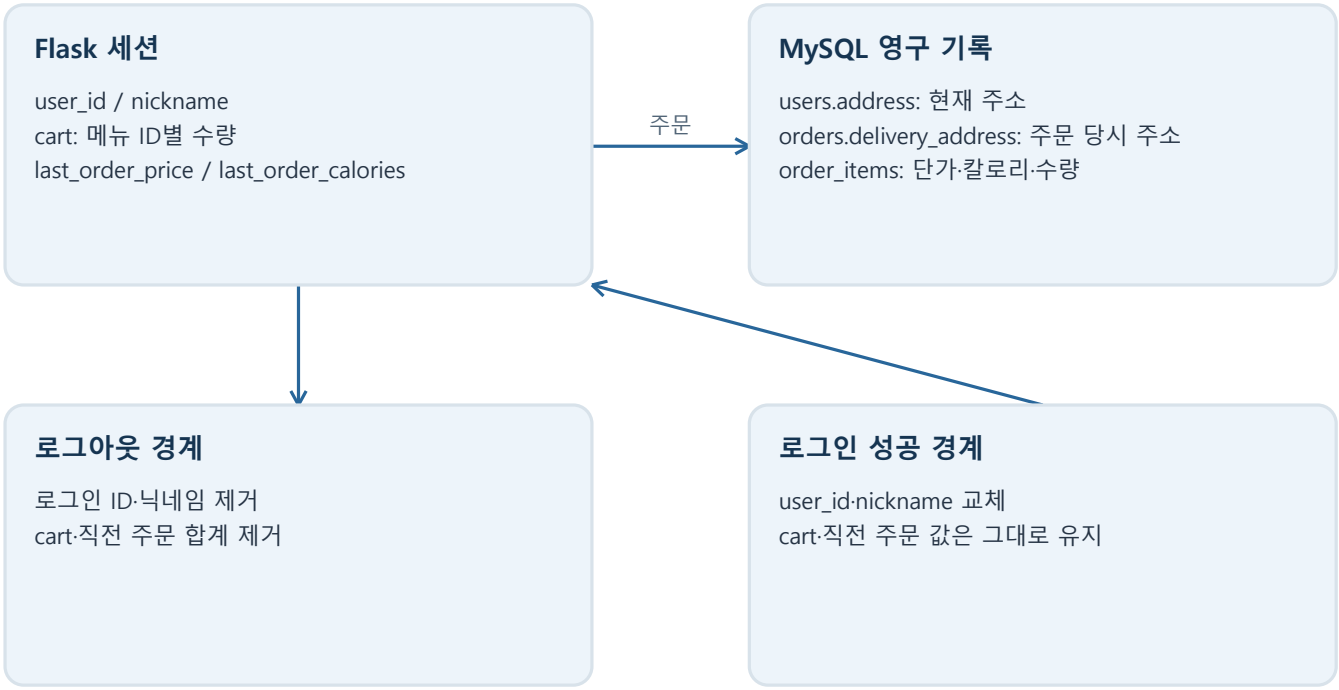
홈 → 음식점 상세 → 장바구니 → 주문 완료 → 주문내역 → 주문 상세 → 배달현황
홈 ↔ 로그인·가입 / 주소 설정 / 절약 통계

메서드·URL	화면 또는 처리 결과	기능
GET /	index.html	음식점 목록·검색
GET /restaurant/<id>	restaurant.html	음식점·메뉴
GET/POST /signup	signup.html	회원가입
GET/POST /login	login.html	로그인
GET /logout	리다이렉트	세션 정리
GET /cart	cart.html	장바구니
POST /cart/add	/cart 이동	수량 증가
POST /cart/minus	/cart 이동	수량 감소
POST /cart/delete	/cart 이동	항목 삭제
POST /order	/order/complete 이동	가상 주문 저장
GET /order/complete	order_complete.html	직전 주문 결과
GET /orders	orders.html	본인 주문 목록
GET /orders/<id>	order_detail.html	본인 주문 상세
GET /saving	saving.html	누적 절약 통계
GET/POST /address	address.html	기본 배달 주소
GET /delivery/<id>	delivery_status.html	가상 배달·지도·거리

<id>는 해당 라우트의 int 타입 restaurant_id 또는 order_id를 줄여 표기했다. GET/POST는 두 메서드 모두 정의된 경로다. 로그인 제한·소유권 검사 여부는 분석 문서의 예외 처리표를 참조한다.

세션·영속 데이터의 경계

그림 6. 클라이언트 세션과 관계형 데이터 저장을 구분한다.



로그아웃 경계와 달리 로그인 성공 경계에서는 장바구니와 직전 합계가 제거되지 않는다. 이는 계정별 영구 장바구니 설계가 아닌 현재 구현의 제약이다.

5. 주문 스냅샷의 적용 범위

데이터	저장 시점·위치	이후 변경의 영향
배달 주소	주문 시 users.address → orders.delivery_address	회원 기본 주소 변경과 독립
단가·칼로리	주문 시 menus → order_items	메뉴의 현재 가격·칼로리 변경과 독립
음식점·메뉴명	상세 조회 시 현재 테이블 JOIN	이름의 변경 이력을 보존하지 않음
장바구니·좌표	cart는 세션; 좌표는 브라우저 메모리	별도 영속 테이블·기기 간 동기화 없음

6. 음식점 조회 식별 기준

DB/restaurant_query.sql은 orders를 포함한 4개 테이블의 전체 주문 조회 예시다. 실제 라우트는 본인 orders를 먼저 확인한 뒤 order_items·menus·restaurants를 JOIN한다. DISTINCT 대상은 r.name, r.address이므로 이름·주소가 같은 서로 다른 ID가 한 결과로 합쳐질 수 있다.

초기 DDL의 FK 4개와 논리적 관계는 일치한다. 운영 DB 적용 여부와 추가 주소 컬럼의 자료형은 미확인이다.